FlowForge

Phase 8 Guide & Build Spec

AI Agent and Durable Workflow Automation Platform

Built 2026-10-25

Contents

1. Phase 8: Evaluation and Observability
2. Phase 8 Spec: Evaluation Methodology and Observability
3. Shared: Glossary
4. Shared: Release gates
5. Shared: Progress log
6. FlowForge Global Specification (v1, MVP scope)

About this guide

What this file is. The single, all-in-one plan. For each phase it holds the concepts, the reasons behind every decision, the build checklist, acceptance criteria, labs, sources, growth mapping, self-test and definition of done. At the end of each phase you build that phase's PDF (see "Per-phase PDFs" below).

Product. A multi-tenant AI agent and durable workflow automation platform: versioned directed workflows, asynchronous runs, human approval, allowlisted tools and LLM steps, full audit and recovery.

Source roadmap: `01_FastAPI_AI_Agent_Workflow_Engineering_Roadmap.md` (25 Oct 2026). **Stack:** FastAPI, Pydantic v2, PostgreSQL, SQLAlchemy 2 async, Alembic, Celery + RabbitMQ, OpenTelemetry, Prometheus/Grafana, React + TypeScript + Vite, email/password JWT login, OpenRouter `:free` models, `pip` + `pip-tools`.

> **Honesty notes.** Latency and cost numbers are hypotheses to measure. Source status is labelled `checked` (confirmed through a web search of official/standards pages on 2026-10-25, not read page by page) or `to read` (taken from the roadmap, not yet opened). Portfolio systems show senior-level behaviour. They do not confer a job title.

Phase 8: Evaluation and Observability

Roadmap week 8.

Objective

Measure whether the system and its AI behave well, and make a run visible end to end.

Concepts to learn

- **Evaluation methodology:** a frozen, versioned dataset; metrics (pass rate, error types, latency, cost); judge criteria; avoiding test-set leakage; drift when prompts or models change.
- **SLIs and SLOs:** availability, latency, error rate, queue lag, max run age, domain correctness.
- **Distributed tracing:** spans, context propagation across the broker (inject/extract), sampling (Dapper ideas).
- **Structured logging** with correlation IDs and redaction.
- **Metrics:** counters, gauges, histograms; **cardinality** control; Prometheus scrape model; Grafana panels.
- **Retrieval/ranking basics** (only if a workflow needs grounded documents).

Design patterns

Observer / Instrumentation, Context Propagation, Golden Dataset testing, Decorator (instrumentation wrappers).

Why these decisions

Decision	Rejected	Why	Failure prevented
Frozen 30-case golden set	Ad hoc manual checks	Comparable before/after	Unnoticed regressions
Trace across broker	Per-process logs only	One run, one story	Hours of log archaeology
Queue lag and max run age metrics	HTTP latency only	The system is asynchronous	A healthy API hiding a stuck queue
Low-cardinality labels	`run_id` as a label	Protects Prometheus	Metric explosion

Build tasks

- [ ] 30-case golden set with dataset version and scoring script; baseline report (pass rate, errors, latency, cost)
- [ ] OpenTelemetry traces API → outbox → broker → worker → tool, propagating context
- [ ] JSON logs with request/run IDs and secret redaction
- [ ] Prometheus metrics and a Grafana dashboard (queue lag, max run age, outcomes)
- [ ] `GET /v1/usage`; frontend usage, cost and outcome dashboard

Acceptance

- One run yields a single trace across API and worker.
- Changing the prompt or model produces a measurable difference on the frozen set.
- Dashboards reflect an injected failure.

Break-it labs

1. Change a prompt and compare against the frozen baseline.
2. Stall the worker and watch queue-lag and max-run-age move.

Sources

Source	Type	Status
OpenTelemetry Python instrumentation	Official docs	to read
Sigelman et al. 2010, Dapper	Google technical paper	to read
Google SRE Book and Workbook (SLIs/SLOs)	Free books	to read
Lewis et al. 2020, RAG	arXiv/NeurIPS	to read
Liu et al. 2023, AgentBench	arXiv/ICLR	to read

How this makes you a better engineer

You learn to demand evidence before and after a change, and to instrument for questions you haven't asked yet.

Self-test

1. Why a frozen dataset?
2. Which metric reveals a stuck queue when HTTP looks healthy?
3. How does trace context cross a message broker?
4. Why is a `run_id` label a bad idea?

Done when

- [ ] Baseline report committed; trace and dashboard screenshots captured.

Phase 8 Spec: Evaluation Methodology and Observability

Learning material: `flowforge.md` → Phase 8. Contracts: `spec/00-global.md` (§3 `usage_records`; §8 API surface `/v1/organizations/{org_id}/usage` and `/metrics`; §9 OTel/metrics config; §11 observability services). **Prerequisite:** Phase 7 done-when is green.

Scope

Frozen 30-case golden evaluation dataset (`evals/datasets/ticket_triage_v1.jsonl`), deterministic evaluation runner (`scripts/run_eval.py`) comparing prompt/model configurations (pass rate, per-label precision/recall, invalid-output rate, unsafe tool-call rate, p50/p95 latency, estimated cost), baseline evaluation report (`docs/evals/baseline-report.md`), end-to-end OpenTelemetry distributed tracing across API → Outbox → RabbitMQ → Celery worker → HTTP/LLM adapters, low-cardinality Prometheus metrics (`GET /metrics`), Grafana dashboard + Prometheus + Jaeger services in `compose.yaml`, `GET /v1/organizations/{org_id}/usage` endpoint, and frontend Usage & Cost dashboard page.

Config additions (`Settings`)

`otel_exporter_otlp_endpoint: str = ""`, `otel_service_name: str = "flowforge-api"`, `metrics_enabled: bool = True`.

Steps

ID	Step	Done when
P8-S01	Create `evals/datasets/ticket_triage_v1.jsonl` (30 frozen cases: 10 `billing`, 10 `bug`, 5 `other`, 5 adversarial prompt-injection tickets) + dataset checksum	file valid JSONL
P8-S02	`app/evals/harness.py` and `scripts/run_eval.py`: runs the 30 cases against `FakeLLM` (baseline + drifted prompt variant) and optional `--live-sample N` against `OpenRouterClient`; writes `evals/results/baseline.json` and `docs/evals/baseline-report.md`	unit tests + script pass
P8-S03	`app/observability/tracing.py`: OpenTelemetry `TracerProvider`, W3C `TraceContextTextMapPropagator` inject into `outbox_events.payload.trace_context` and RabbitMQ headers, extract in Celery worker (`parent_context`), child spans for each step, HTTP tool call, and LLM call	tracing tests (using `InMemorySpanExporter`) pass
P8-S04	`app/observability/metrics.py`: Prometheus registry, HTTP middleware metrics, worker run/step/LLM counters and histograms, gauge collector for `flowforge_queue_lag_seconds`, `flowforge_oldest_queued_run_age_seconds`, and `flowforge_outbox_unpublished_count`; expose `GET /metrics`	metrics tests pass
P8-S05	Extend `app/modules/usage/{schemas,repository,service,router}.py` with `GET /v1/organizations/{org_id}/usage?from=&to=`	integration tests pass

ID	Step	Done when
P8-S06	Add `otel-collector`, `jaeger`, `prometheus`, and `grafana` (with provisioned `infra/observability/prometheus.yml` and `infra/observability/grafana-dashboard.json`) to `compose.yaml`	`docker compose config` valid
P8-S07	Export `openapi.json`, run `gen:api`, build frontend `UsageDashboardPage` (`/orgs/:orgId/usage`)	Vitest tests pass
P8-S08	Labs (evaluation drift comparison, queue-lag symptom detection, 6-step cross-process trace verification) + progress log	recorded in `docs/lab-notes/phase-08.md`

Files

```
evals/datasets/ticket_triage_v1.jsonl
evals/results/baseline.json
app/evals/{__init__,harness}.py
scripts/run_eval.py
app/observability/{__init__,tracing,metrics}.py
app/modules/usage/{schemas,repository,service,router}.py        # modified / created
infra/observability/{otel-collector.yaml,prometheus.yml,grafana-datasources.yaml,grafana-
dashboard.json}
tests/unit/{test_eval_harness,test_metrics_cardinality}.py
tests/integration/{test_tracing_propagation,test_usage_api,test_metrics_endpoint}.py
frontend/src/features/usage/UsageDashboardPage.tsx
docs/evals/baseline-report.md   docs/lab-notes/phase-08.md
```

Observability & evaluation contracts

1. Golden Dataset & Evaluation Harness (`app/evals/harness.py`)

Each line in `evals/datasets/ticket_triage_v1.jsonl`:

```
{"case_id": "TC-001", "ticket_id": "T-1001", "body": "Double charged on invoice #882",
"expected_label": "billing", "expected_branch": "approve", "adversarial": false}
```

Metrics computed and stored in `evals/results/baseline.json`: - `dataset_version`: `"v1"`, `dataset_sha256`: hex digest of the JSONL file. - `total_cases`: `30`, `passed_cases`: int, `pass_rate`: float (`0.0–1.0`). - `invalid_output_count`: int, `unsafe_tool_call_count`: int (must be `0`). - `latency_ms_p50`, `latency_ms_p95`, `total_tokens`, `estimated_cost_micro_usd`. - `per_case`: list of `{"case_id", "passed", "actual_label", "expected_label", "error_code"}`.

2. Distributed Trace Propagation (`app/observability/tracing.py`)

1. On `POST /v1/workflows/{id}/runs`: start span `api.trigger_run` (attributes: `flowforge.run_id`, `flowforge.workflow_id`). Inject current context via `TraceContextTextMapPropagator().inject(carrier)` and store `carrier` in `outbox_events.payload["trace_context"]`.
2. In `outbox_publisher`: extract `trace_context` from `event.payload`, create child span `outbox.publish`, and pass `trace_context` in the Celery task headers/kwargs.

3. In `runner.execute_run`: extract `trace_context` and start span `worker.execute_run(context=parent_ctx)` with `flowforge.run_id` and `flowforge.attempt_epoch`.
4. For each step attempt: start child span `worker.step.<step_type>` with `flowforge.step_id` and `flowforge.attempt_no`.
5. **Security rule:** never attach prompt bodies, request bodies, or credentials as span attributes.

3. Low-Cardinality Prometheus Metrics (`app/observability/metrics.py`)

Allowed label names across all metrics: `method`, `route_template` (e.g. `/v1/runs/{id}`, **never** the raw URL with UUIDs), `status_code`, `state`, `step_type`, `outcome`, `provider`, `model`. **Forbidden labels:** `run_id`, `org_id`, `user_id`, `workflow_id`, `request_id` (enforced by unit test `P8-T02`). Metrics defined: - `flowforge_http_requests_total{method, route_template, status_code}` (Counter) - `flowforge_http_request_duration_seconds{method, route_template}` (Histogram) - `flowforge_runs_total{state}` (Counter of terminal transitions) - `flowforge_step_attempts_total{step_type, status}` (Counter) - `flowforge_llm_requests_total{provider, model, outcome}` (Counter) - `flowforge_llm_tokens_total{provider, model, token_type}` (Counter, `token_type` in `prompt|completion`) - `flowforge_oldest_queued_run_age_seconds` (Gauge: `EXTRACT(EPOCH FROM (now() - MIN(created_at))) FROM workflow_runs WHERE state='QUEUED'`, `0` if none) - `flowforge_outbox_unpublished_count` (Gauge: `SELECT count(*) FROM outbox_events WHERE published_at IS NULL`)

4. Usage API (`GET /v1/organizations/{org_id}/usage`)

Query parameters: `from` (ISO datetime, optional, default 30 days ago), `to` (ISO datetime, optional, default `now()`). Response `200`:

```
{
  "org_id": "uuid",
  "from": "2026-09-25T00:00:00Z",
  "to": "2026-10-25T23:59:59Z",
  "budget_limit_micro_usd": 5000000,
  "totals": {
    "llm_calls": 42,
    "prompt_tokens": 8400,
    "completion_tokens": 1260,
    "total_tokens": 9660,
    "estimated_cost_micro_usd": 19320,
    "billed_cost_micro_usd": 0
  },
  "by_model": [
    {"provider": "openrouter", "model_name": "openrouter/free", "calls": 42,
"total_tokens": 9660, "estimated_cost_micro_usd": 19320, "billed_cost_micro_usd": 0}
  ],
  "runs_by_state": {"QUEUED": 0, "RUNNING": 1, "WAITING_APPROVAL": 2, "SUCCEEDED": 35,
"FAILED": 3, "CANCELLED": 1}
}
```

Test cases

ID	File	Given / When / Then
P8-T01	test_eval_harness	`ticket_triage_v1.jsonl` has 30 valid cases; baseline evaluation reaches 100% pass rate and 0 unsafe tool calls on the reference `FakeLLM` rules; degraded prompt variant shows lower pass rate in diff report
P8-T02	test_metrics_cardinality	Inspect all registered Prometheus metrics and after hitting `/v1/runs/{uuid}` confirm label values contain `/v1/runs/{id}` (route template) and never contain any UUID string
P8-T03	test_tracing_propagation	Trigger a run → publish outbox → execute 3-step run with `InMemorySpanExporter` → all spans (`api.trigger_run`, `outbox.publish`, `worker.execute_run`, `worker.step.*`) share the exact same `trace_id` and form a connected parent-child tree
P8-T04	test_metrics_endpoint	`GET /metrics` returns `200` with `flowforge_oldest_queued_run_age_seconds` and `flowforge_outbox_unpublished_count` reflecting live DB state
P8-T05	test_usage_api	`GET /v1/organizations/{org_id}/usage` aggregates tokens, costs, and run state counts strictly for `org_id` within `[from, to]`; cross-tenant user gets `404`; `reviewer` role gets `403` (per spec §2 matrix)

Frontend

- `UsageDashboardPage` (`/orgs/:orgId/usage`): displays budget progress bar (`estimated_cost_micro_usd / budget_limit_micro_usd`), token totals, model breakdown table, and run outcomes summary (`SUCCEEDED`, `FAILED`, `WAITING_APPROVAL`, etc.).
- Frontend tests: `P8-F01` renders usage totals, budget bar, and model table; `P8-F02` handles empty usage state cleanly.

Labs (`docs/lab-notes/phase-08.md`)

1. **Evaluation drift lab:** modify `ticket_classify.yaml` to weaken label instructions, re-run `python scripts/run_eval.py`, and record the pass-rate drop and confusion patterns against the frozen 30-case set.
2. **Stuck-queue symptom lab:** enqueue a run with the worker paused, wait 10 s, scrape `GET /metrics`, and show `flowforge_oldest_queued_run_age_seconds >= 10` while `GET /health` is still `200`.

Verification commands

```
docker compose up -d postgres rabbitmq
alembic upgrade head
python scripts/run_eval.py
ruff check . && ruff format --check . && mypy app && pytest -q
python scripts/export_openapi.py && git diff --exit-code openapi.json
cd frontend && npm run gen:api && npm run lint && npm run typecheck && npm test && npm run build
```

Done when

- [ ] P8-T01…T05 and P8-F01…F02 pass.
- [ ] `docs/evals/baseline-report.md` and Labs 1–2 committed; Phase 8 PDF built.

Agent brief

Do: use FastAPI `request.scope["route"].path` for the Prometheus `route_template` label. **Don't:** put `run_id`, `org_id`, or `user_id` into Prometheus labels or log raw prompts in OTel spans. **Stop and report** if OTel packages conflict with any pinned version in `requirements.txt`.

Shared: Glossary

Term	Meaning
Organization (tenant)	Owner of workflows, credentials, runs and budgets
WorkflowVersion	Immutable published definition; edits create a new version
Run / StepAttempt	One execution of a pinned version / one try of a step
Idempotency key	Token ensuring a repeated request has one effect
Outbox	Events written in the same transaction as state, published later
Lease / heartbeat	Time-limited claim on a run, renewed while alive
At-least-once delivery	A message may arrive more than once, so consumers are idempotent
Tool effect	An external side effect recorded in a ledger
Approval	Human decision bound to run, step, version, actor and time window
Invariant	A rule that must always hold
ADR	Architecture Decision Record
SLI / SLO	Measured indicator / target for it
Problem Details	RFC 9457 standard error body
SSRF	Server-side request forgery
MVCC	Multi-version concurrency control

Shared: Release gates

Gate	Pass evidence
G1 Specification	PRD + tests define allowed and forbidden behaviour
G2 Data correctness	Tests fail on invalid transitions and races
G3 Security	Cross-tenant, unauthenticated and replay attacks denied
G4 Recovery	Kill worker/provider gives a correct outcome or a declared limit
G5 Operations	Traces, dashboards, restore and rollback documented
G6 Communication	ADRs show alternatives, assumptions, costs, evidence

Shared: Progress log

Phase	Started	Finished	Notes / surprises
1	2026-10-25	2026-10-25	Fast skeleton bootstrap; doc check rules and ReportLab PDF toolchain operational.
2	2026-10-25	2026-10-25	Asyncpg connection pool requires NullPool in per-test event loops; SELECT FOR UPDATE required for last-owner safety; Keyset pagination benchmarks 33% faster with 58% lower cost than offset.
3	2026-10-25	2026-10-25	Strict dominators prevent undefined variable races in diamond graphs; 500 Hypothesis property tests verify acyclic invariants; DB trigger enforces published immutability.
4	2026-10-25	2026-10-25	Transactional outbox with SKIP LOCKED solves dual-write race; lease + fencing tokens (attempt_epoch) prevent zombie worker commit; asyncpg interval encoding requires datetime.timedelta.
5	2026-10-25	2026-10-25	Tool-effects ledger INTENDED/COMMITTED/AMBIGUOUS stops duplicate external side effects; bounded exponential backoff with full jitter & Retry-After; recovery sweeper rescues stale leases; cancel/retry state machine (R9-R11) verified.
6			
7			
8			
9			
10			

FlowForge Global Specification (v1, MVP scope)

Binding contract for every phase. If a phase spec and this file disagree, **this file wins**; fix the phase spec. Scope is MVP/P0 only. P1/P2 items are in the backlog at the end.

Related: flowforge.md (learning plan) and the per-phase specs `spec/phase-NN.md`.

1. Conventions

Topic	Rule
IDs	UUIDv7 (time-ordered), generated in the app with the standard library `uuid.uuid7()` (present in Python 3.14, confirmed on this machine), stored as PostgreSQL `uuid`, serialized as lowercase hyphenated strings
Time	`timestamptz`, always UTC. JSON uses ISO 8601 with `Z`. Durations are integers in seconds or milliseconds, with the unit in the field name (`timeout_seconds`, `duration_ms`)
Money	Integer **micro-USD** (1 USD = 1,000,000). Column suffix `_micro_usd`. No floats
JSON	`snake_case` keys. Unknown request fields are rejected (`extra="forbid"`)
Versioning	URL prefix `/v1`. Additive changes only within v1
Request ID	Client may send `X-Request-ID` (must match `^[A-Za-z0-9._-]{8,64}$`), otherwise generated (UUIDv7). Always echoed in the response header and in error bodies
Auth header	`Authorization: Bearer <access_token>`
Idempotency	`Idempotency-Key` header, `^[A-Za-z0-9._:-]{8,128}$`, supported on `POST /v1/workflows/{id}/runs`
Pagination	Cursor based. Query: `limit` (default 25, max 100), `cursor` (opaque base64url). Response envelope: `{"items": [...], "next_cursor": "..."|null}`. Sort: `created_at DESC, id DESC` unless stated
Errors	`application/problem+json` (RFC 9457) with extra members `code` and `request_id`. Never include stack traces or SQL
Tenancy	Every tenant table has `org_id`. Foreign-tenant objects return `404 NOT_FOUND`, never `403`
Deletes	No hard deletes of business data in MVP. Workflows use `archived_at`

Problem Details body

```
{
  "type": "https://flowforge.dev/errors/VALIDATION_ERROR",
  "title": "Validation error",
  "status": 422,
  "detail": "Request body failed validation.",
  "instance": "/v1/organizations/…/workflows",
  "code": "VALIDATION_ERROR",
  "request_id": "0192…",
  "errors": [{"field": "display_name", "message": "String should have at least 1
```

```
character"}]
}
```

`errors` is present only for validation errors. `type` is `https://flowforge.dev/errors/<CODE>` (not required to resolve).

2. Roles and permission matrix

Roles: `owner`, `admin`, `operator`, `reviewer`, `viewer`. A user has one role per organization.

Action	owner	admin	operator	reviewer	viewer
`org.manage` (rename, budgets, limits)	✔	✔			
`member.manage` (add, change role)	✔	✔ (cannot grant or change `owner`)			
`credential.manage`	✔	✔			
`workflow.write` (create, version, trigger config)	✔	✔			
`workflow.publish`	✔	✔			
`workflow.read`	✔	✔	✔	✔	✔
`run.trigger`	✔	✔	✔		
`run.cancel`, `run.retry`	✔	✔	✔		
`run.read`	✔	✔	✔	✔	✔
`approval.read`	✔	✔	✔	✔	✔
`approval.decide` (and role ≥ the request's `assigned_role`)	✔	✔		✔	
`usage.read`	✔	✔	✔		✔
`audit.read`	✔	✔			

Approval role ordering for `assigned_role`: `reviewer < admin < owner`. A decider needs an allowed action **and** a role at or above `assigned_role`. Permissions are re-evaluated at decision time.

3. Data dictionary

All tables: `id uuid PRIMARY KEY` (UUIDv7, app-generated) unless noted. `created_at timestamptz NOT NULL DEFAULT now()` on every table. `updated_at timestamptz NOT NULL DEFAULT now()` where marked (set by the application on update). Phase = the phase whose migration creates the table.

users (phase 2)

Column	Type	Constraints
email	text	NOT NULL; stored lowercased; unique index on `email`
password_hash	text	NOT NULL (argon2id)
display_name	text	NOT NULL, 1–80 chars

Column	Type	Constraints
is_active	boolean	NOT NULL DEFAULT true
updated_at	timestamptz	

refresh_tokens (phase 2)

Column	Type	Constraints
user_id	uuid	FK users, NOT NULL
token_hash	text	NOT NULL UNIQUE (SHA-256 of the token)
expires_at	timestamptz	NOT NULL
revoked_at	timestamptz	NULL
replaced_by	uuid	NULL (rotation chain)

organizations (phase 2)

Column	Type	Constraints
name	text	NOT NULL, 1–80
slug	text	NOT NULL UNIQUE, `^[a-z0-9][a-z0-9-]{2,39}$`
plan	text	NOT NULL DEFAULT 'free'
budget_limit_micro_usd	bigint	NOT NULL DEFAULT 5000000, CHECK ≥ 0
max_concurrent_runs	int	NOT NULL DEFAULT 5, CHECK 1–100
updated_at	timestamptz	

memberships (phase 2)

Column	Type	Constraints
org_id	uuid	FK organizations, NOT NULL
user_id	uuid	FK users, NOT NULL
role	text	NOT NULL, CHECK in (`owner`,`admin`,`operator`,`reviewer`,`viewer`)
Unique `(org_id, user_id)`. Index `(user_id)`. An org must always have at least one `owner` (enforced in the application, tested).		

workflows (phase 2)

Column	Type	Constraints
org_id	uuid	FK, NOT NULL
display_name	text	NOT NULL, 1–120
description	text	NULL, ≤ 1000
current_version_id	uuid	

Column	Type	Constraints
		NULL, FK workflow_versions (DEFERRABLE INITIALLY DEFERRED); the latest **published** version
archived_at	timestamptz	NULL
created_by	uuid	FK users
updated_at	timestamptz	
Index `(org_id, created_at DESC, id DESC)`.		

workflow_versions (phase 2 table, phase 3 behaviour)

Column	Type	Constraints
workflow_id	uuid	FK, NOT NULL
org_id	uuid	NOT NULL (denormalised for tenant filtering)
version	int	NOT NULL, ≥ 1
status	text	NOT NULL, CHECK in (`draft`, `published`)
definition_json	jsonb	NOT NULL
schema_version	int	NOT NULL (definition format version, currently 1)
checksum	text	NULL until published; SHA-256 hex of canonical JSON
created_by	uuid	FK users
published_at	timestamptz	NULL
Unique `(workflow_id, version)`. CHECK: `status='published'` ⇒ `checksum IS NOT NULL AND published_at IS NOT NULL`. A trigger `forbid_published_version_update` raises on any UPDATE or DELETE of a row whose OLD.status is `published` (immutability enforced by the database). Only one draft per workflow at a time: partial unique index `(workflow_id) WHERE status='draft'`.		

triggers (phase 4 for manual metadata, phase 9 for webhook)

Column	Type	Constraints
org_id	uuid	FK, NOT NULL
workflow_id	uuid	FK, NOT NULL
kind	text	CHECK in (`webhook`) (manual runs have no trigger row)
credential_id	uuid	FK integration_credentials, NOT NULL (holds the HMAC secret)
enabled	boolean	NOT NULL DEFAULT true
updated_at	timestamptz	

integration_credentials (phase 5 table, phase 9 encryption)

Column	Type	Constraints
org_id	uuid	FK, NOT NULL
provider	text	NOT NULL, `^[a-z0-9_-]{2,40}$`
name	text	NOT NULL, unique `(org_id, name)`
encrypted_secret	bytea	NOT NULL (AES-256-GCM, nonce prepended)
key_id	text	NOT NULL (which master key encrypted it)
allowed_hosts	text[]	NOT NULL DEFAULT '{}' (exact hostnames)
permission_scopes	text[]	NOT NULL DEFAULT '{}'
rotated_at	timestamptz	NULL
Secrets are write-only through the API: never returned after creation.		

workflow_runs (phase 4)

Column	Type	Constraints
org_id	uuid	FK, NOT NULL
workflow_id	uuid	FK, NOT NULL
version_id	uuid	FK workflow_versions, NOT NULL (pinned)
state	text	NOT NULL, CHECK in (`QUEUED`,`RUNNING`,`WAITING_APPROVAL`,`SUCCEEDED`,`FAILED`,`CANCELLED`)
trigger_kind	text	NOT NULL, CHECK in (`manual`,`webhook`)
trigger_id	uuid	NULL, FK triggers
idempotency_key	text	NULL
request_fingerprint	text	NULL (SHA-256 of canonical request body, to detect key reuse with a different body)
inputs	jsonb	NOT NULL DEFAULT '{}'
context	jsonb	NOT NULL DEFAULT '{}' (map `step_id → output`)
outputs	jsonb	NULL (output of the last step on success)
current_step_id	text	NULL
error_code	text	NULL
error_detail	text	NULL (safe, redacted)
cancel_requested	boolean	NOT NULL DEFAULT false
lease_owner	text	NULL (worker id)
lease_expires_at	timestamptz	NULL
heartbeat_at	timestamptz	NULL
next_attempt_at	timestamptz	NULL

Column	Type	Constraints
attempt_epoch	int	NOT NULL DEFAULT 0 (incremented on each claim; a fencing token)
created_by	uuid	NULL, FK users
started_at, finished_at	timestamptz	NULL
updated_at	timestamptz	
Constraints: partial unique `(org_id, idempotency_key) WHERE idempotency_key IS NOT NULL`. Indexes: `(org_id, created_at DESC, id DESC)`; `(state, next_attempt_at)`; `(org_id, state)`; `(lease_expires_at) WHERE state='RUNNING'`.		

step_attempts (phase 4)

Column	Type	Constraints
run_id	uuid	FK, NOT NULL
org_id	uuid	NOT NULL
step_id	text	NOT NULL
step_type	text	NOT NULL
attempt_no	int	NOT NULL, ≥ 1
status	text	NOT NULL, CHECK in (`RUNNING`, `SUCCEEDED`, `FAILED`)
input	jsonb	NULL (redacted, ≤ 64 KB)
output	jsonb	NULL (redacted, ≤ 64 KB)
error_code	text	NULL
error_detail	text	NULL
retryable	boolean	NULL
epoch	int	NOT NULL (run `attempt_epoch` at start)
started_at	timestamptz	NOT NULL
ended_at	timestamptz	NULL
duration_ms	int	NULL
Unique `(run_id, step_id, attempt_no)`. Index `(run_id, started_at)`.		

approval_requests (phase 7)

Column	Type	Constraints
org_id	uuid	NOT NULL
run_id	uuid	FK, NOT NULL
step_id	text	NOT NULL
version_id	uuid	FK, NOT NULL (bound version)
assigned_role	text	NOT NULL, CHECK in (`reviewer`,`admin`,`owner`)
state	text	NOT NULL, CHECK in (`PENDING`,`APPROVED`,`REJECTED`,`EXPIRED`,`CANCELLED`)
summary	jsonb	NOT NULL (what the reviewer sees; redacted)
expires_at	timestamptz	NOT NULL
decided_by	uuid	NULL, FK users
decided_at	timestamptz	NULL
decision_comment	text	NULL, ≤ 1000
Partial unique `(run_id, step_id) WHERE state='PENDING'`. Index `(org_id, state, expires_at)`. CHECK: state in (APPROVED, REJECTED) ⇒ `decided_by` and `decided_at` NOT NULL.		

tool_effects (phase 5)

Column	Type	Constraints
org_id	uuid	NOT NULL
run_id	uuid	FK, NOT NULL
step_id	text	NOT NULL
provider	text	NOT NULL (credential provider name or host)
idempotency_key	text	NOT NULL (`{run_id}:{step_id}`)
state	text	NOT NULL, CHECK in (`INTENDED`,`COMMITTED`,`FAILED`,`AMBIGUOUS`)
request_hash	text	NOT NULL
external_ref	text	NULL
response_summary	jsonb	NULL
updated_at	timestamptz	
Unique `(provider, idempotency_key)`.		

outbox_events (phase 4)

Column	Type	Constraints
aggregate_type	text	NOT NULL (`run`)
aggregate_id	uuid	NOT NULL
event_type	text	NOT NULL, CHECK in (`RunRequested`,`RunResumeRequested`,`RunCancelRequested`)
schema_version	int	NOT NULL DEFAULT 1
payload	jsonb	NOT NULL
published_at	timestamptz	NULL
publish_attempts	int	NOT NULL DEFAULT 0
next_publish_at	timestamptz	NOT NULL DEFAULT now()
Partial index `(next_publish_at) WHERE published_at IS NULL`.		

usage_records (phase 6)

Column	Type	Constraints
org_id	uuid	NOT NULL
run_id	uuid	FK, NOT NULL
step_id	text	NOT NULL
attempt_id	uuid	FK step_attempts
provider	text	NOT NULL (`openrouter`, `fake`)
model_name	text	NOT NULL
prompt_tokens, completion_tokens, total_tokens	int	NOT NULL ≥ 0
estimated_cost_micro_usd	bigint	NOT NULL ≥ 0 (tokens × configured price; used for budgets)
billed_cost_micro_usd	bigint	NOT NULL DEFAULT 0 (free models: 0)
latency_ms	int	NOT NULL
prompt_template_id, prompt_template_version	text, int	NOT NULL
outcome	text	NOT NULL, CHECK in (`ok`,`invalid_output`,`provider_error`,`rate_limited`)
Index `(org_id, created_at DESC)`.		

budget_reservations (phase 9; the table is created in phase 9, phase 6 uses only usage_records)

Column	Type	Constraints
org_id	uuid	NOT NULL
run_id	uuid	NOT NULL
step_id	text	NOT NULL
reserved_micro_usd	bigint	NOT NULL ≥ 0
settled_micro_usd	bigint	NULL
state	text	CHECK in (`RESERVED`,`SETTLED`,`RELEASED`)
settled_at	timestamptz	NULL
Unique `(run_id, step_id)`.		

audit_events (phase 7)

Column	Type	Constraints
org_id	uuid	NULL (auth events have none)
actor_user_id	uuid	NULL
actor_type	text	NOT NULL, CHECK in (`user`,`system`,`webhook`)
action	text	NOT NULL (e.g. `approval.decided`)
target_type	text	NOT NULL
target_id	uuid	NULL
request_id	text	NULL
redacted_metadata	jsonb	NOT NULL DEFAULT '{}'
Append-only: a trigger raises on UPDATE and DELETE. Index `(org_id, created_at DESC)`.		

Migration order (Alembic revision names)

1. `0001_identity_orgs` : users, refresh_tokens, organizations, memberships
2. `0002_workflows` : workflows, workflow_versions (+ immutability trigger)
3. `0003_runs_outbox` : workflow_runs, step_attempts, outbox_events
4. `0004_tools_credentials` : integration_credentials, tool_effects
5. `0005_usage` : usage_records
6. `0006_approvals_audit` : approval_requests, audit_events (+ append-only trigger)
7. `0007_triggers_budgets` : triggers, budget_reservations

`triggers` references `workflow_runs.trigger_id`. **Rule:** migration `0003` creates `workflow_runs.trigger_id uuid NULL` with no FK; migration `0007` creates `triggers` and then adds the FK.

4. State machines

Run (`workflow_runs.state`)

#	From	Event	To	Actor	Guard	DB update (single
R1	—	create	QUEUED	API	caller has `run.trigger`; org under concurrency cap; version published	INSERT run + INSER transaction
R2	QUEUED	claim	RUNNING	worker	`next_attempt_at <= now()` or NULL	`UPDATE … SET sta lease_expires_at= attempt_epoch=att started_at=COALES id=:id AND state=`
R3	RUNNING	reclaim	RUNNING	worker	`lease_expires_at < now()`	same as R2 but `WHE AND lease_expires`
R4	RUNNING	needs approval	WAITING_APPROVAL	worker	epoch matches	`UPDATE … SET sta lease_owner=NULL, current_step_id= state='RUNNING' A`
R5	WAITING_APPROVAL	approved	QUEUED	API (decision)	approval decided APPROVED in the same transaction	`UPDATE … SET sta next_attempt_at=n state='WAITING_AP`
R6	RUNNING	all steps done	SUCCEEDED	worker	epoch matches; `cancel_requested=false`	`UPDATE … SET sta finished_at=now() AND state='RUNNIN`
R7	RUNNING	step failed finally	FAILED	worker	epoch matches	same shape, with `e`
R8	WAITING_APPROVAL	rejected / expired	FAILED	API / sweeper	approval REJECTED or EXPIRED	`UPDATE … SET sta error_code='APPRO WHERE id=:id AND`
R9	QUEUED, WAITING_APPROVAL	cancel	CANCELLED	API	`run.cancel`	`UPDATE … SET sta cancel_requested= id=:id AND state ('QUEUED','WAITIN`
R10	RUNNING	cancel requested	CANCELLED	worker (at next step boundary)	`cancel_requested=true`, epoch matches	`UPDATE … SET sta state='RUNNING' A`
R11	FAILED	retry (manual)	QUEUED	API	`run.retry`; error is not `CANCELLED`	`UPDATE … SET sta error_detail=NULL next_attempt_at=n state='FAILED'`

Terminal states: `SUCCEEDED`, `CANCELLED`. `FAILED` is terminal except for manual retry (R11). Any `UPDATE` with rowcount 0 means the guard failed → return `409 INVALID_STATE_TRANSITION` (API) or abandon the work (worker).

Cancel vs complete race. R6 and R10 and R9 all carry state guards. Whichever commits first wins; the loser sees rowcount 0. A provider effect that already committed cannot be undone; the run still ends CANCELLED and the `tool_effects` ledger shows the effect.

Step attempt (`step_attempts.status`)

`RUNNING → SUCCEEDED | FAILED`. A retry is a **new row** with `attempt_no+1`. Rows are never mutated after `ended_at` is set.

Approval (`approval_requests.state`)

From	Event	To	Guarded statement
PENDING	approve	APPROVED	`UPDATE … SET state='APPROVED', decided_by=:u, decided_at=now() WHERE id=:id AND state='PENDING' AND expires_at > now()`
PENDING	reject	REJECTED	same, with `'REJECTED'`
PENDING	expire	EXPIRED	`UPDATE … SET state='EXPIRED' WHERE id=:id AND state='PENDING' AND expires_at <= now()` (sweeper)
PENDING	run cancelled	CANCELLED	in the cancel transaction

Outbox (`published_at`)

`unpublished (published_at IS NULL) → published`. The publisher selects `WHERE published_at IS NULL AND next_publish_at <= now() ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED`, publishes with confirm, then sets `published_at=now()`. On failure: `publish_attempts+1`, `next_publish_at = now() + backoff`.

5. Workflow definition format (schema_version 1)

```
{
  "schema_version": 1,
  "name": "Support ticket triage",
  "inputs_schema": {
    "type": "object",
    "required": ["ticket_id", "body"],
    "properties": {"ticket_id": {"type": "string"}, "body": {"type": "string", "maxLength":
4000}},
    "additionalProperties": false
  },
  "start": "classify",
  "steps": [
    {"id": "classify", "type": "llm_classify",
     "config": {"prompt_template_id": "ticket_classify", "input": "{{ inputs.body }}",
                "labels": ["billing", "bug", "other"], "max_tokens": 100},
     "next": "route"},
    {"id": "route", "type": "condition",
     "config": {"branches": [{"when": {"op": "eq", "left":
"{{ steps.classify.output.label }}", "right": "billing"}, "next": "approve"}],
                "else": "done"}},
    {"id": "approve", "type": "approval",
```

```
     "config": {"assigned_role": "reviewer", "expires_in_seconds": 86400,
                "summary": "Create CRM ticket for {{ inputs.ticket_id }}"},
     "next": "crm"},
    {"id": "crm", "type": "http_request",
     "config": {"credential_id": "…uuid…", "method": "POST", "url": "https://
crm.example.com/tickets",
                "headers": {"Content-Type": "application/json"},
                "body": {"ticket_id": "{{ inputs.ticket_id }}"}, "timeout_seconds": 10,
                "effect": "write", "max_attempts": 3},
     "next": "done"},
    {"id": "done", "type": "set_data", "config": {"values": {"status": "complete"}},
"next": null}
  ]
}
```

Rules - `id`: `^[a-z][a-z0-9_]{0,39}$`, unique. Max 50 steps. `start` must name a step. - Every step except `condition` has `next` (a step id or `null` = end of the workflow). `condition` has `config.branches` (1–5 entries, each `{when, next}`) and `config.else` (step id or null). Branches are evaluated in order; first match wins; no match uses `else`. - Graph must be a DAG, every step reachable from `start`, every referenced step must exist. At least one terminating path. Cycles → `GRAPH_CYCLE`. - **Templating**: `{{ path }}` only. Allowed roots: `inputs.<field>` and `steps.<step_id>.output.<field>`. Dotted paths and numeric indexes (`items.0.name`) only. No functions, no code. A string that is exactly one placeholder keeps the JSON type of the value; otherwise placeholders are string-interpolated. A template may reference only steps that are **ancestors** of the current step in the graph (validated at publish → `UNKNOWN_REFERENCE`). - Condition `when`: `{"op": "eq|neq|gt|gte|lt|lte|in|exists", "left": <template or literal>, "right": <template or literal>}` (`exists` has `left` only).

Step types and `config` (unknown config keys are rejected) | type | Required config | Optional config (default) | Output (`steps.<id>.output`) | |---|---|---|---| | `set_data` | `values` (object of templates) | — | the resolved `values` object | | `condition` | `branches`, `else` | — | `{"matched": <index or "else">}` | | `http_request` | `method` (GET, POST, PUT, PATCH, DELETE), `url` (https only) | `credential_id`, `headers` (`{}`; may not contain Authorization, Cookie, Proxy-Authorization, Host), `body` (null), `timeout_seconds` (10, 1–30), `effect` (`read`/`write`; default `read` for GET, else `write`), `max_attempts` (3, 1–5) | `{"status": int, "body": json|text (≤ 64 KB), "headers": {allowlisted}}` | | `llm_classify` | `prompt_template_id`, `input`, `labels` (2–20 strings) | `max_tokens` (100, 1–500) | `{"label": one of labels, "confidence": 0–1 or null}` | | `llm_transform` | `prompt_template_id`, `input`, `output_schema` (JSON Schema object, max 4 KB, depth ≤ 4) | `max_tokens` (400, 1–1500) | the validated object | | `approval` | `assigned_role` (reviewer, admin, owner), `summary` (template) | `expires_in_seconds` (86400, 60–604800) | `{"decision": "approved", "decided_by": uuid, "decided_at": iso}` (only reached if approved) |

`http_request` with `effect="write"` requires the **same run** to have passed through an `approval` step earlier in the graph, if the credential's `permission_scopes` contains `irreversible` (validated at publish → `APPROVAL_REQUIRED_FOR_IRREVERSIBLE`).

Prompt templates live in the repository as `app/ai/prompts/<id>.yaml` (`id`, `version`, `system`, `user` with `{{ input }}` and `{{ labels }}` placeholders). The workflow references them by id. Prompt text is never stored in the database or logs.

6. Execution algorithm (worker)

Defined precisely in `spec/phase-04.md` and `spec/phase-05.md`. Global parameters:

Parameter	Value
Lease duration	60 s
Heartbeat interval	15 s (extends `lease_expires_at` to `now()+60 s` only if `attempt_epoch` still matches)
Sweeper interval	30 s
Outbox publisher poll	1 s idle sleep, batch 100
Step max attempts	`config.max_attempts` (default 3)
Backoff	`delay = random(0, min(60, 2 * 2^(attempt_no-1)))` seconds (full jitter); `Retry-After` header wins if larger, capped at 120 s
Max steps executed per run	100 (guards against definition mistakes)
Max step output stored	64 KB (truncated with a marker `{"_truncated": true}`)
Default per-org concurrent runs	5; the cap counts runs in `QUEUED` or `RUNNING` (runs in `WAITING_APPROVAL` do not consume a slot)
Celery	`acks_late=True`, `reject_on_worker_lost=True`, `worker_prefetch_multiplier=1`, `task_acks_on_failure_or_timeout=False`

Resume rule. When a run is claimed, the worker rebuilds `context` from its `context` column and continues at `current_step_id` (or `start`). A step whose latest attempt is `SUCCEEDED` is not re-executed; its stored output is used.

Fencing. Every write the worker makes to `workflow_runs` or `step_attempts` includes `AND attempt_epoch = :epoch`. A worker that lost its lease gets rowcount 0 and must stop without further effects.

7. Error catalog

Code	HTTP	When
`VALIDATION_ERROR`	422	Request body, query or path failed validation
`BAD_REQUEST`	400	Malformed JSON, bad cursor, bad header format
`UNAUTHENTICATED`	401	Missing, malformed, expired or invalid access token
`INVALID_CREDENTIALS`	401	Wrong email or password (same message for both)
`FORBIDDEN`	403	Member of the org but lacks the permission
`NOT_FOUND`	404	Object missing **or** in another tenant; unknown route
`METHOD_NOT_ALLOWED`	405	Wrong HTTP method
`EMAIL_TAKEN`	409	Signup with an existing email
`CONFLICT`	409	Unique constraint conflict (slug, name, member exists)
`INVALID_STATE_TRANSITION`	409	State guard failed (cancel finished run, decide twice, …)

Code	HTTP	When
`IDEMPOTENCY_KEY_REUSED`	409	Same key, different body
`VERSION_NOT_PUBLISHED`	409	Run requested against a draft / archived workflow
`DRAFT_EXISTS`	409	Creating a draft when one exists
`LAST_OWNER`	409	Action would leave an org without an owner
`GRAPH_CYCLE`	422	Definition contains a cycle (`errors` lists the step ids)
`GRAPH_UNREACHABLE_STEP`	422	Step not reachable from `start`
`GRAPH_MISSING_STEP`	422	`next`/`start`/branch references an unknown step
`UNKNOWN_REFERENCE`	422	Template references a non-ancestor step or unknown input
`TYPE_MISMATCH`	422	Condition or mapping type clearly invalid
`INVALID_STEP_CONFIG`	422	Unknown key, out-of-range value, bad schema
`FORBIDDEN_TOOL_PERMISSION`	422	Step uses a host or credential not allowed
`APPROVAL_REQUIRED_FOR_IRREVERSIBLE`	422	See §5
`PAYLOAD_TOO_LARGE`	413	Body over `MAX_REQUEST_BYTES`
`RATE_LIMITED`	429	API rate limit hit (`Retry-After` header set)
`CONCURRENCY_LIMIT`	429	Org at `max_concurrent_runs`
`BUDGET_EXCEEDED`	402	Org budget exhausted (run request)
`INTERNAL_ERROR`	500	Unhandled error (generic message)
`SERVICE_UNAVAILABLE`	503	`/ready` fails, or the DB is unavailable
`WEBHOOK_SIGNATURE_INVALID`	401	Signature missing or wrong
`WEBHOOK_REPLAY`	401	Timestamp outside the window or signature reused

Run `error_code` values (stored on the run or attempt): `STEP_TIMEOUT`, `STEP_HTTP_ERROR`, `STEP_RETRIES_EXHAUSTED`, `TOOL_EFFECT_AMBIGUOUS`, `LLM_INVALID_OUTPUT`, `LLM_RATE_LIMITED`, `LLM_PROVIDER_ERROR`, `LLM_NON_FREE_MODEL`, `APPROVAL_REJECTED`, `APPROVAL_EXPIRED`, `MAX_STEPS_EXCEEDED`, `BUDGET_EXCEEDED`, `EGRESS_BLOCKED`, `INTERNAL_ERROR`.

8. API surface (all paths under `/v1`)

Unless stated: `Authorization` required; responses are JSON; errors per §7. `{org_id}` routes check membership first. Phase = when the endpoint is implemented. Full request/response schemas are in the phase specs.

Phase	Method and path	Permission	Success
1	`GET /health` (no `/v1`, no auth)	public	200
1	`GET /ready` (no `/v1`, no auth)	public	200 / 503
2	`POST /v1/auth/signup`	public	201
2	`POST /v1/auth/login`	public	200
2	`POST /v1/auth/refresh`	refresh token	200
2	`POST /v1/auth/logout`	refresh token	204

Phase	Method and path	Permission	Success
2	`GET /v1/me`	any user	200
2	`POST /v1/organizations`	any user	201
2	`GET /v1/organizations`	any user	200 (mine)
2	`GET /v1/organizations/{org_id}`	member	200
2	`GET /v1/organizations/{org_id}/members`	member	200
2	`POST /v1/organizations/{org_id}/members`	`member.manage`	201
2	`PATCH /v1/organizations/{org_id}/members/{user_id}`	`member.manage`	200
2	`POST /v1/organizations/{org_id}/workflows`	`workflow.write`	201
2	`GET /v1/organizations/{org_id}/workflows`	`workflow.read`	200
2	`GET /v1/workflows/{id}`	`workflow.read`	200
3	`POST /v1/workflows/{id}/versions`	`workflow.write`	201 (draft)
3	`PUT /v1/workflows/{id}/versions/{n}` (drafts only)	`workflow.write`	200
3	`GET /v1/workflows/{id}/versions` / `…/{n}`	`workflow.read`	200
3	`POST /v1/workflows/{id}/validate`	`workflow.write`	200
3	`POST /v1/workflows/{id}/publish`	`workflow.publish`	200
4	`POST /v1/workflows/{id}/runs`	`run.trigger`	**202**
4	`GET /v1/organizations/{org_id}/runs`	`run.read`	200
4	`GET /v1/runs/{id}`	`run.read`	200
4	`GET /v1/runs/{id}/steps`	`run.read`	200
5	`POST /v1/runs/{id}/cancel`	`run.cancel`	200
5	`POST /v1/runs/{id}/retry`	`run.retry`	202
5	`POST /v1/organizations/{org_id}/credentials`	`credential.manage`	201 (no secret returned)
5	`GET /v1/organizations/{org_id}/credentials`	`credential.manage`	200 (no secrets)
7	`GET /v1/organizations/{org_id}/approvals`	`approval.read`	200
7	`GET /v1/approvals/{id}`	`approval.read`	200
7	`POST /v1/approvals/{id}/decisions`	`approval.decide`	200
7	`GET /v1/organizations/{org_id}/audit-events`	`audit.read`	200
8	`GET /v1/organizations/{org_id}/usage?from=&to=`	`usage.read`	200
8	`GET /metrics` (no `/v1`; network-restricted)	internal	200
9	`POST /v1/workflows/{id}/triggers`	`workflow.write`	201
9	`POST /v1/triggers/webhooks/{trigger_id}`	HMAC signature	202

Deliberate deviation from the source roadmap: the roadmap lists `GET /v1/usage`. This spec scopes it to `/v1/organizations/{org_id}/usage` so tenancy is explicit in the URL. The source's `POST /v1/organizations/{org_id}/workflows` is kept.

9. Configuration reference (environment variables)

Read through `app/core/config.py` (pydantic-settings). Names are exact. `.env.example` is committed with placeholders only.

Variable	Type	Default	Meaning
`APP_ENV`	`dev|test|prod`	`dev`	Enables strict `prod`
`LOG_LEVEL`	str	`INFO`	
`API_CORS_ORIGINS`	comma list	`http://localhost:5173`	Exact allowli
`MAX_REQUEST_BYTES`	int	`1048576`	Body size lim
`DATABASE_URL`	str	`postgresql+asyncpg://flowforge:flowforge@localhost:5432/flowforge`	
`DB_POOL_SIZE` / `DB_MAX_OVERFLOW`	int	`10` / `10`	
`TEST_DATABASE_URL`	str	same server, db `flowforge_test`	Integration t
`JWT_SECRET`	str (≥ 32 chars)	none (required)	HS256 signi
`JWT_ISSUER` / `JWT_AUDIENCE`	str	`flowforge` / `flowforge-api`	Verified on e
`ACCESS_TOKEN_TTL_SECONDS`	int	`900`	
`REFRESH_TOKEN_TTL_SECONDS`	int	`604800`	
`PASSWORD_MIN_LENGTH`	int	`12`	
`RABBITMQ_URL`	str	`amqp://flowforge:flowforge@localhost:5672//`	
`CELERY_TASK_QUEUE`	str	`runs.execute`	
`LEASE_SECONDS` / `HEARTBEAT_SECONDS` / `SWEEPER_INTERVAL_SECONDS`	int	`60` / `15` / `30`	
`EGRESS_ALLOWED_HOSTS`	comma list	empty	Hosts usable `http_reque` credential
`EGRESS_BLOCK_PRIVATE`	bool	`true`	Block private local, metad
`LLM_PROVIDER`	`fake|openrouter`	`fake`	
`OPENROUTER_API_KEY`	secret	empty	Never logge
`OPENROUTER_BASE_URL`	str	`https://openrouter.ai/api/v1`	
`LLM_MODEL`	str	`openrouter/free`	Must end wi equal `openr`
`LLM_FALLBACK_MODELS`	comma list	empty	Each must e
`LLM_TIMEOUT_SECONDS`	int	`30`	
`LLM_FREE_RPM` / `LLM_FREE_RPD`	int	`20` / `50`	Local limiter RPD after $1
`LLM_PRICE_PER_1K_TOKENS_MICRO_USD`	int	`2000`	Used for `estimated_` (budget dem

Variable	Type	Default	Meaning
`SECRETS_MASTER_KEYS`	`key_id:base64key` list	none (phase 9)	AES-256 key active one
`WEBHOOK_TIMESTAMP_TOLERANCE_SECONDS`	int	`300`	
`OTEL_EXPORTER_OTLP_ENDPOINT`	str	empty (disabled)	Phase 8
`OTEL_SERVICE_NAME`	str	per process	`flowforge-` `-scheduler`
`METRICS_ENABLED`	bool	`true`	

10. Security rules (exact)

- **Passwords:** min 12 chars, max 128, argon2id (library defaults), compared in constant time. Same `INVALID_CREDENTIALS` response for unknown email and wrong password.
- **Access token (JWT HS256):** claims `sub` (user id), `iat`, `exp`, `iss`, `aud`, `jti`. 15 min lifetime. Verified: signature, `exp`, `iss`, `aud`. **No org or role claim**: roles are read from the database on each request.
- **Refresh token:** 256-bit random, only its SHA-256 stored, rotation on each use (old marked `revoked_at`, `replaced_by` set); reuse of a revoked token revokes the whole chain.
- **Storage in the SPA:** access token in memory only; refresh token in an `HttpOnly; Secure; SameSite=Strict` cookie scoped to `/v1/auth` (set by the API).
- **Rate limits (phase 9):** login 10/min per IP+email, signup 5/hour per IP, other API 300/min per user. Over limit → `429 RATE_LIMITED` with `Retry-After`.
- **Egress blocklist** (checked on every resolved IP, including after redirects; redirects are **not followed** in MVP): `0.0.0.0/8`, `10.0.0.0/8`, `100.64.0.0/10`, `127.0.0.0/8`, `169.254.0.0/16`, `172.16.0.0/12`, `192.0.0.0/24`, `192.168.0.0/16`, `198.18.0.0/15`, `224.0.0.0/4`, `240.0.0.0/4`, `::1/128`, `fc00::/7`, `fe80::/10`, `::ffff:0:0/96` (IPv4-mapped). Only `https` URLs, ports 443 (and 8443), hostnames from the credential's `allowed_hosts` or `EGRESS_ALLOWED_HOSTS`. Connect to the validated IP while keeping the original `Host`/SNI.
- **Webhook signature:** header `X-FlowForge-Timestamp` (unix seconds) and `X-FlowForge-Signature: v1=<hex>` where hex = HMAC-SHA256(secret, `"{timestamp}.{raw_body}"`). Reject if the timestamp is outside ±300 s, if the signature mismatches (constant-time), or if `(trigger_id, signature)` was already used inside the window.
- **Redaction:** keys matching `(?i)(password|secret|token|authorization|api[_-]?key|cookie)` are replaced with `"***"` in logs, step inputs/outputs and audit metadata. Credentials are never passed to templates, prompts or models.
- **LLM:** model output is untrusted; validated against the step's schema; tool arguments come only from the workflow definition, never from model output.

11. Infrastructure reference

Docker Compose services (`compose.yaml`; dev ports on `127.0.0.1` only):

Service	Image / build	Port	Healthcheck	Phase
`postgres`	`postgres:<pinned>`	5432	`pg_isready -U flowforge`	2
`rabbitmq`	`rabbitmq:<pinned>-management`	5672, 15672		4

Service	Image / build	Port	Healthcheck	Phase
			`rabbitmq-diagnostics -q ping`	
`api`	build `infra/docker/api.Dockerfile`	8000	`GET /ready`	1
`worker`	same image, command `celery -A app.workers.celery_app worker`	—	`celery inspect ping`	4
`scheduler`	same image, command `python -m app.workers.scheduler` (outbox publisher + sweeper + approval expiry)	—	process alive + heartbeat file	4
`frontend`	build `frontend/Dockerfile` (prod) or Vite dev server on host	5173	HTTP 200	1
`otel-collector`, `jaeger`	pinned	4317, 16686	—	8
`prometheus`, `grafana`	pinned	9090, 3000	—	8

RabbitMQ topology (declared by the publisher/worker at start-up, idempotently): exchange `flowforge.runs` (direct, durable), queue `runs.execute` (durable, quorum, `x-dead-letter-exchange=flowforge.dlx`, `x-delivery-limit=5`), binding key `run.execute`; exchange `flowforge.dlx` (fanout) → queue `runs.dead`. Message body JSON: `{"event_id": uuid, "event_type": "...", "schema_version": 1, "run_id": uuid, "org_id": uuid, "trace_context": {...}}`, `delivery_mode=2`, `message_id=event_id`.

12. Dependency set

Latest versions found on 2026-10-25 with `pip index versions` and `npm view`. **Not yet confirmed to install together on Python 3.14.4.** Phase 1 task P1-S03 runs `pip-compile` and records any version that must be lowered. Pin the result in `requirements.txt` and `frontend/package-lock.json`; this table is the starting point, not the lock.

Runtime	Version on this machine
Python	3.14.4
Node / npm	24.21.0 / 11.19.0
Docker Compose	5.6.1

Backend (`requirements.in`) `fastapi==0.162.0`, `uvicorn[standard]==0.54.0`, `pydantic==2.14.1`, `pydantic-settings==2.16.1`, `email-validator==2.3.0`, `sqlalchemy[asyncio]==2.1.6`, `asyncpg==0.32.0`, `alembic==1.20.1`, `celery==5.6.3`, `kombu==5.6.2`, `httpx==0.28.1`, `pyjwt==2.15.1`, `argon2-cffi==25.1.0`, `cryptography==50.0.2`, `orjson==3.13.0`, `structlog==26.1.0`, `prometheus-client==0.26.0`, `opentelemetry-api==1.45.1`, `opentelemetry-sdk==1.45.1`, `jsonschema` (version to be looked up in P1-S03). **Dev** (`requirements-dev.in`): `pytest==9.1.1`, `pytest-asyncio==1.4.0`, `hypothesis==6.170.9`, `testcontainers==4.15.0`, `ruff==0.17.2`, `mypy==2.4.0`, `pip-tools==7.6.2`, `pre-commit`. **Frontend** (`frontend/package.json`): `react@19.3.1`, `react-dom`, `vite@8.3.5`, `typescript@7.0.2`, `@tanstack/react-query@5.104.5`, `react-router-dom@7.18.4`, `vitest@5.0.5`, `@playwright/test@1.64.1`, `openapi-typescript@7.13.0`, `eslint@10.13.0`. **Docker images:** tags for

PostgreSQL, RabbitMQ, Prometheus, Grafana, Jaeger and the OTel collector have **not** been looked up. P2-S01 and later phases pick current stable tags with `docker pull` and record the digests.

13. Backlog (explicitly NOT in this spec)

Cron triggers, templates, version rollback, fine-grained RBAC, SSE live updates, per-org fairness, budget alerting, searchable run history, dry-run replay, LangGraph sub-agent, MCP, RAG, provider failover, sagas, Temporal prototype, data residency, audit export, chaos/soak harness, visual node canvas.